HARMONIA DSL and Its Simulation Systems

A technical and conceptual overview of the language, `.hrm` programs, swarm dynamics, ANIMUS, and EEG-informed experimentation

Prepared for: Andrew Kadziolka

Drafted by: Manus AI

Date: July 26, 2026

Evidence base: `Andrewkadz/HARMONIA-DSL`, revision `7612f925`

> **Scope note.** HARMONIA is best described at present as a research language family and simulation laboratory. The repository contains working interpreters and simulations, but it does not yet expose one canonical compiler, one uniform `.hrm` dialect, or one experimentally validated theory of intelligence. This report therefore distinguishes **implemented software**, **documented design intent**, **author-provided project history**, and **interpretive claims**.

Executive summary

HARMONIA DSL is an experimental domain-specific language and runtime ecosystem for representing adaptive systems in terms of coupled internal dimensions, symbolic operators, feedback, memory, regulation, and collective coherence. Its central design idea is that a system should not be described only by conventional control flow. It should also carry an explicit state for factors such as awareness or phase (`Ψ` / `psi`), value or structural orientation (`Φ` / `phi`), disorder or uncertainty (`ε` / `epsilon`), energy, memory, and coherence. Programs can then express transformations and constraints over those dimensions.[1] [2]

The repository implements this idea through several related but distinct paths. One path is a symbolic Φπε interpreter with glyph-like operators. Another is a nine-layer numerical integration stack. A third treats `.hrm` files as declarative "harmonic scores" that describe systems, dimensions, laws, and constraints. Several simulation engines then apply HARMONIA concepts to harmonic swarms, cluster scheduling, Tau-crystal processing, quantum-inspired agents, lattice dynamics, games, and neural or EEG-oriented experiments.[3] [4]

The most concrete swarm baseline is `pure_harmonic_swarm.py`. It models a population of mathematical agents whose states evolve through coupled numerical updates. These are not automatically language-model agents and should not be described as independent artificial minds. They are dynamical agents carrying variables such as `psi`, `phi`, `omega`,

`epsilon`, energy, velocity, entropy, knowledge, and maturity. The collective can use global mean-field or local-neighborhood coupling, and the current implementation reports a coherence statistic defined as `1 / (1 + std(phi))`.[5]

ANIMUS is the project's regulatory interpretation of collective coherence. In the cluster simulations, measured coherence is translated into a scheduling or concurrency policy: when the simulated collective becomes less coherent, an external control layer can restrict activity; when coherence is sufficient, the system can permit greater expression or throughput. This makes ANIMUS less a separate intelligence than a **policy interface between observed collective state and system-level resource control**.[6][7]

The project author reports that **EEG data was used to train or condition the swarm**. The repository includes named utilities for EEG analysis, conversion to HARMONIA representations, neural bridging, and EEG-driven swarm simulation, supporting the existence of an EEG-to-HARMONIA experimental track.[13][14][15][16] This report preserves the author's historical attribution while using "training" cautiously: without a separately verified loss function, optimization procedure, held-out evaluation, and provenance record, the strongest objective wording is that EEG-derived data was used as an empirical source for **parameterization, conditioning, calibration, or training experiments** involving the swarm. No clinical, diagnostic, consciousness, or brain-decoding claim follows from that fact alone.

1. What HARMONIA DSL is

A domain-specific language is a language designed around a particular class of problems. HARMONIA's chosen problem is the description of systems whose behavior depends on interacting internal states rather than only on variables and functions in the conventional sense. Its vocabulary combines symbolic notation, dynamical systems, feedback regulation, memory, ethics or value orientation, uncertainty, and collective behavior.[1][2]

The repository repeatedly uses the dimensions `Ψ`, `Φ`, and `ε`. Their exact labels vary somewhat across documents and runtimes, but the broad pattern is stable:

Dimension	Common repository interpretation	Practical software role
`Ψ` / `psi`	Awareness, phase, state, or intention	An agent-level evolving variable, often phase-like
`Φ` / `phi`	Ethics, structure, value, position, or integration	An agent-level orientation or position used in coupling and coherence

`ε` / `epsilon`	Entropy, disorder, noise, or uncertainty	A limiting or perturbing quantity
`ω` / `omega`	Frequency, change, or coherence-related motion	A rate or oscillator parameter in some runtimes
`Σ` / `sigma`	Integrated or expressed state	A combined output or stability quantity

One documented formulation is:

```
Σ = (Ψ + Φ) × (1 − ε)
```

This expresses a general HARMONIA motif: increasing disorder or uncertainty limits the expression of the other dimensions. The formula functions as a conceptual stabilizing law in documentation and parts of the symbolic runtime; it should not be assumed to be the sole update equation used by every simulation. [1] [2]

The project's philosophical framing includes the author's description of HARMONIA as **primarchical and archetypal**. That framing describes the intended character and origin of the system. It is not itself a directly testable software property, so an objective account should present it as authorial theory rather than as an empirical result.

2. HARMONIA is a family of runtimes, not one pipeline

The repository contains several generations of implementation. They share terminology and goals, but they are not yet fully unified.

Runtime family	Primary role	Current relationship to other families
Symbolic Φπε interpreter	Executes symbolic operators over a structured state	A substantial standalone runtime with compatibility helpers
Nine-layer GHE integrator	Passes state through memory, energy, intention, growth, coupling, and dynamics layers	A separate numerical integration path
`.hrm` harmonic-score runner	Loads a declarative score and instantiates a harmonic swarm	The clearest direct `.hrm`-to-swarm path, but only partially interprets rule bodies

Legacy parser/compiler paths	Parse or translate other Harmonia-like syntaxes	Not consistently connected to the primary swarm runner
Specialized simulations	Explore cluster, Tau, quantum-inspired, lattice, game, and neural applications	Share concepts but frequently use their own state models and assumptions
Interface layer	IDEs, shells, servers, dashboards, and frontends	Provides several experimental ways to interact with the ecosystem

This multiplicity is important. It shows that HARMONIA has been explored across several computational forms, but it also creates ambiguity about which syntax and semantics are canonical. A user can encounter multiple meanings for the same symbol and multiple parsers for files described as HARMONIA programs. A future stabilization effort should define a versioned language core, a canonical abstract syntax tree, one reference runtime, and explicit compatibility boundaries.

3. The symbolic Φπε language

The symbolic path is centered on `phi_pi_e_interpreter.py`. It maintains an explicit state and dispatches a vocabulary of glyph-like operators. The broader language references describe operations associated with dimensions such as `Φ`, `Π`, `E`, `ε`, `Δ`, `Ψ`, `Λ`, `Ξ`, and `ω`.[2][8]

The symbolic form is significant because it treats a Harmonia program as more than a configuration file. Operators can transform state, combine observations, regulate expression, and invoke higher-order processes. This path is the closest part of the repository to a conventional interpreter for a symbolic DSL.

At the same time, the symbolic interpreter and the `.hrm` swarm runner are not one continuous compiler pipeline. A symbolic program written for the Φπε interpreter should not be assumed to execute through `harmonia_runner.py`, and an `.hrm` law should not be assumed to invoke the full symbolic operator set. They currently represent related dialects within the same research ecosystem.

4. `.hrm` files: harmonic scores

The `.hrm` file type is the project's strongest declarative program format. It is described as a **harmonic score**: a file that defines a system, its dimensions, its laws, its constraints, and in some dialects its inputs, outputs, agents, and state transitions.[3]

A representative score is `cluster_rescue.hrm`:

```
Plain Text
```

```
SYSTEM ClusterRescue {
    AGENTS: 100
    TOPOLOGY: MeanField
    DT: 0.05
}

DIMENSIONS {
    PSI: 1.0
    PHI: 1.0
    OMEGA: 0.0
}

LAW "ThermalThrottling" {
    INPUT: Temperature
    IF Temperature > 80:
        PERTURB PSI BY (Temperature - 80) * 0.02
}

CONSTRAINT "AnimusGate" {
    WHEN COHERENCE < 0.8:
        EMIT "THROTTLE_ACTIVE"
        SET_GLOBAL MAX_CONCURRENCY = 20
}
```

This file communicates an intelligible experiment: create a 100-agent mean-field system, initialize its dimensions, expose it to a temperature-related law, and apply a coherence gate to concurrency. 9

4.1 What the current `.hrm` runner actually does

`harmonia_runner.py` reads selected `SYSTEM` and `DIMENSIONS` fields, captures named `LAW` and `CONSTRAINT` blocks, creates a `HarmonicSwarm`, applies inputs, advances the numerical engine, calculates statistics, and emits events or global values. 4

However, the runner is currently a prototype rather than a general compiler for arbitrary rule bodies. Its Python code recognizes particular law and constraint names—such as thermal throttling, schism injection, ANIMUS gating, and healing—and then executes hard-coded behavior for those names. Changing a mathematical expression inside an `.hrm` body does not necessarily change runtime behavior unless the Python mapping is also updated. 4

This distinction is central to an objective description:

> An `.hrm` file presently acts as a partially executable declarative score. Some configuration is parsed generically, while important law and constraint behavior remains

> name-dispatched in Python.

5. The harmonic swarm

The harmonic swarm is the repository’ s foundational collective-dynamics engine. It creates many lightweight agents and updates them repeatedly through a coupled numerical process. 5

5.1 Agent state

The implementation exposes state variables that can be measured across the collective:

Variable	Functional interpretation in the implementation
`psi`	Phase-like internal state
`phi`	Position or structural state used by the implemented coherence statistic
`omega`	Frequency or rate term
`epsilon`	Noise, exploration, or uncertainty term
`energy`	Derived energetic state
`velocity`	Rate of positional change
`entropy`	Derived disorder-related state
`knowledge`	Accumulated internal quantity
`maturity`	Derived developmental quantity

The engine supports global mean-field coupling and an optional local interaction radius. In global mode, each agent responds to a collective field. In local mode, it responds to a neighborhood. The updates combine oscillatory or phase behavior, damping, coupling, and stochastic variation. 5

5.2 What coherence means in this engine

The current `get_statistics()` implementation defines coherence as:

```
Plain Text
```

```
coherence = 1 / (1 + standard_deviation(phi))
```

The value approaches `1` as the spread of `phi` approaches zero. It declines as the population becomes more dispersed.[5]

This statistic is useful, but its name should be qualified. It is specifically a **position-coherence proxy based on `phi` dispersion**. It is not automatically equivalent to agreement, truth, intelligence, safety, or phase synchronization. A separate phase-order statistic based on `psi` is needed if the experiment intends to measure oscillator synchronization.

A common phase measure is conceptually:

Plain Text

```
phase_order = |mean(exp(i × psi_j))|
```

This distinction matters because a population can remain tightly grouped in one state variable while becoming dispersed in another. That disagreement can be more informative than a single composite score.

5.3 What the successful 20-step run established

The locally executed command was:

Bash

```
python3 harmonia_runner.py cluster_rescue.hrm --steps 20
```

It loaded `ClusterRescue`, instantiated 100 agents, completed all 20 steps, held the displayed temperature at `50.0`, and reported coherence declining gently from approximately `1.000` to `0.996`. No `THROTTLE_ACTIVE` event appeared.

That result establishes that the selected score and runtime can execute. It does **not** establish whether the swarm is intelligent, robust, useful, trained, or superior to a control. The original CLI prints a narrow smoke-test trace; it does not automatically export a reproducibility manifest, full telemetry, final population state, comparative control, or statistical assessment.

6. What the swarm simulation is for

The strongest practical description is that the swarm simulation is a **digital wind tunnel for collective dynamics**. It allows a designer to define many interacting units, control how

they influence one another, expose the collective to bounded conditions, and measure what emerges.

For a brain trust such as UOR, the longer-term value would be to examine collective reasoning without assuming that consensus equals correctness. A useful brain-trust model could represent participants or AI agents as holders of observations, hypotheses, confidence, memory, or roles. The system could then study how local exchanges affect collective coverage, convergence, minority retention, polarization, recovery, and decision quality.

Potential use	Evidence needed before claiming value
Distributed exploration	Agents maintain meaningfully different candidate states or strategies
Robustness	Collective performance survives loss or drift of a minority of agents
Adaptive regulation	Measured collective state changes an explicit policy or resource allocation
Emergent coordination	Global order arises from interaction rather than from directly writing the answer into the controller
Brain-trust support	The process preserves relevant dissent, integrates evidence, and improves a defined outcome over a baseline
Collective memory	Historical behavior is compressed without erasing significant novelty or provenance

The swarm is not valuable merely because it contains many agents. A swarm can add cost, correlation, instability, fake diversity, or groupthink. Its value must be demonstrated against an appropriate baseline, such as a single agent, independent agents without coupling, majority voting, or constant-input control.

7. ANIMUS: coherence as a regulatory resource

ANIMUS is one of the project’ s most distinctive system-level ideas. In the cluster-oriented simulations, collective coherence is used to derive a regulatory signal that influences scheduling or concurrency. 6 7

The logic can be summarized as:

```
Plain Text
```

```
agent dynamics
      ↓
collective state
      ↓
coherence measurement
      ↓
ANIMUS regulatory interpretation
      ↓
policy action: throttle, permit, allocate, or heal
```

This separates two responsibilities. The swarm produces and exposes collective state. A deterministic policy layer decides what operational action follows. That is a valuable architectural distinction because the numerical collective is not granted unrestricted authority over the host system.

In `cluster_rescue.hrm`, the declared constraint intends to reduce maximum concurrency when coherence falls below a threshold. The Python runner similarly emits a throttle event and changes a global concurrency value for recognized gate constraints.[4] [9]

ANIMUS should therefore be understood as a **governance and orchestration mechanism**, not as proof of a conscious collective. Its quality depends on the validity of its metrics, thresholds, failure modes, and policy mapping.

8. Simulation families in the repository

HARMONIA contains multiple simulation families that explore different interpretations of coupled intelligence and regulation.

Family	What it explores	Objective status
Pure harmonic swarm	Numerical agents coupled through global or local fields	Concrete executable baseline
`.hrm` cluster rescue	Declarative system configuration, external input, and ANIMUS gate	Executable through a partially hard-coded runner
Cluster and coherent-flow simulations	Coherence-informed scheduling and resource control	Application-level prototypes
Tau-crystal swarm	Structured processing inspired by Tau, crystal, or geometric organization	Specialized experimental engine and documentation

Quantum-inspired swarm	Amplitudes, phases, entanglement-like or quantum-inspired collective operations	Classical software using quantum-inspired concepts unless executed on verified quantum hardware
ΞΛΩΨ lattice	Triadic or lattice-style interactions and multi-layer state propagation	Large specialized prototype
Homeostasis and bifurcation experiments	Stability, regulation, divergence, and recovery	Experimental testbeds
Game-oriented swarms	Coordinated behavior in bounded rule environments	Application prototypes
Neural and EEG-oriented simulations	Mapping neural-derived data into HARMONIA/swarm representations	Experimental data-integration track
LLM bridges and chats	Connecting Harmonia concepts to language-model interaction	Separate from the mathematical baseline swarm

These families demonstrate breadth, but they do not yet constitute a single interchangeable API. Results from one engine should not be generalized to another without showing that their state variables, update laws, inputs, and metrics are equivalent.

9. EEG-informed swarm experimentation

9.1 Project-history statement

The project author states:

> **EEG data was used to train the HARMONIA swarm.**

This is part of the project’ s development history and should be included in an overview. The repository contains EEG- and neural-oriented files named `analyze_eeg.py`, `advance_eeg.py`, `eeg_to_hrm.py`, `simulate_eeg_swarm.py`, `neural_bridge.py`, `NEURAL_INTERFACE.hrm`, and `NCL_NEURAL_MAP.hrm`.[13][14][15][16] Their presence is consistent with an experimental workflow in which EEG-derived information is analyzed, translated into HARMONIA-compatible representations, or used to drive a swarm simulation.

9.2 What “trained” should mean in an objective report

The term **training** can refer to several technically different activities:

Possible meaning	Evidence normally required
Initial-state conditioning	A documented mapping from EEG-derived values to agent state
Parameter calibration	An objective function and procedure for fitting coupling, noise, thresholds, or other parameters
Supervised model training	Inputs, labels, loss function, optimizer, training split, and evaluation split
Online adaptation	A rule showing how incoming EEG-derived features update swarm state over time
Scenario generation	EEG-derived patterns used to construct or replay experimental conditions

Based on the evidence reviewed for this draft, the safest summary is:

> The author used EEG-derived data as an empirical input and training or conditioning source for HARMONIA swarm experiments. The repository contains code and `.hrm` assets for EEG analysis, conversion, neural interfacing, and swarm simulation. The exact training protocol, dataset provenance, optimization objective, and independent evaluation should be documented separately before making stronger performance claims.

This wording neither erases the author’s work nor converts an experimental history into a claim that the repository alone has not yet established.

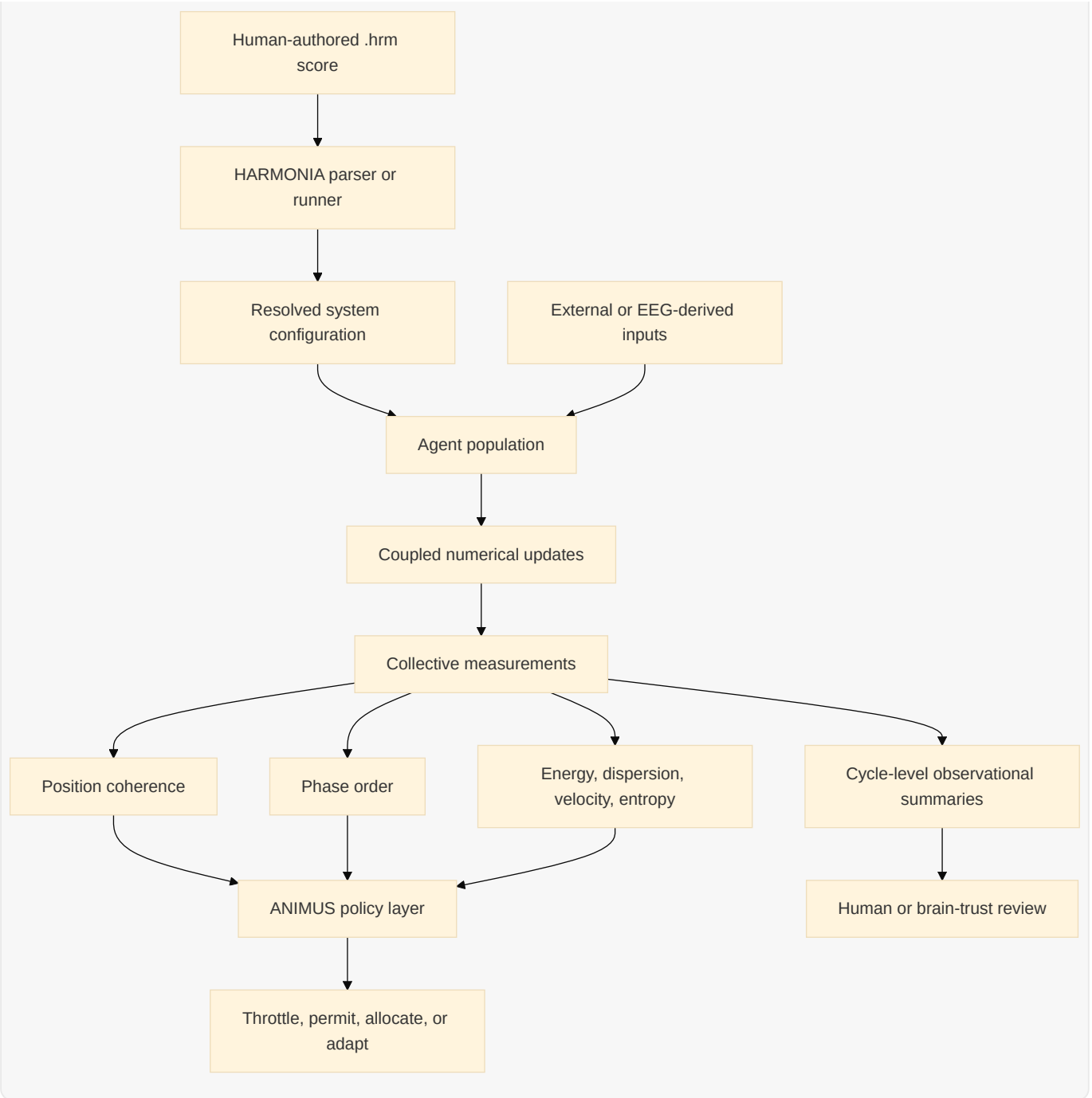
9.3 What the EEG work does not prove by itself

EEG-informed parameterization does not by itself demonstrate that the swarm reproduces a human brain, decodes private mental content, diagnoses a condition, establishes consciousness, or predicts an individual’s behavior. Those would require substantially different evidence and validation. An objective report should also avoid comparing personal EEG data against a normative baseline unless that comparison is explicitly part of a consented study design.

No raw personal EEG data needs to appear in a public overview. The report can describe the pipeline, feature mapping, and experimental role while protecting source-data privacy.

10. Conceptual architecture

```
mermaid
```



This diagram describes the intended relationship among the main concepts. Not every existing runtime implements every arrow.

11. Recommended figures for a complete project file

The following figures would make the overview more useful without overstating the science.

Figure	Required data	What it would show
HARMONIA runtime map	File/module relationships	How symbolic, GHE, `.hrm`, swarm, ANIMUS, EEG, and

		interface paths relate
Coherence versus time	Per-step `phi` dispersion	Whether the collective remains positionally coordinated
Phase order versus time	Per-step `psi` values	Whether oscillator-like states synchronize or diverge
Energy and velocity	Per-step population statistics	Whether excitation accumulates, dissipates, or oscillates
Input and response	Environmental or EEG-derived input beside swarm metrics	Timing, lag, and recovery behavior
Agent-state distribution	Final or sampled population states	Whether a mean conceals clusters, outliers, or polarization
Control-versus-treatment comparison	Same-seed constant and bounded-input runs	What changes are attributable to the experimental condition rather than endogenous drift
Cyclic-truth timeline	One summary per bounded cycle	Compact historical memory that preserves major changes and novelty

A graph is only as meaningful as its labels and provenance. Each figure should state the `.hrm` score, source revision, random seed, agent count, timestep, input profile, number of steps, sampling interval, and metric definition.

12. Cyclic truths and compressed collective memory

The project’ s preferred methodology includes compressing historical observations into **cyclic truths**. In a technically careful formulation, a cyclic truth is a bounded, descriptive summary of one observation cycle. It should not be treated as an immutable or metaphysical truth.

A cycle record might contain:

Field	Purpose
Start and end step	Defines the observation boundary
Input range	Records the condition applied during the cycle

Initial, final, and minimum coherence	Describes stability and recovery
Mean phase order	Describes synchronization during the cycle
Mean and peak dispersion	Reveals spread and outliers
Energy change	Shows accumulation or dissipation
Gate events	Records regulatory actions
Novel observation	Preserves a meaningful change not captured by averages

An example statement is:

> Cycle 2 remained in a high-position-coherence regime; coherence declined slightly and then partially recovered, phase order decreased, and no gate event occurred.

This approach limits storage growth while maintaining raw step telemetry for runs that require detailed analysis. It is especially useful when the system operates continuously and cannot retain every agent emission indefinitely.

13. How to assess a HARMONIA swarm objectively

A defensible assessment should begin with a precise question and a comparison condition. For example:

> Does a 100-agent harmonic swarm preserve coordination and recover its initial state under a smooth, bounded input cycle more effectively than under an uncoupled or alternative-coupling condition?

The experiment should then record:

Category	Minimum measurement
Reproducibility	Score hash, source revision, seed, parameters, environment
Coordination	Position coherence and phase order as separate metrics
Diversity	Population variance, clusters, outliers, or hypothesis coverage
Dynamics	Energy, velocity RMS, entropy, and response lag

Regulation	Gate thresholds, event counts, and policy actions
Recovery	Return error after each bounded cycle
Cost	Agent updates, execution time, and memory where relevant
Comparative value	Difference from an explicit control or simpler baseline

High coherence alone is insufficient. A brain trust can be highly coherent and uniformly wrong. If the swarm is later used for reasoning tasks, it must also be evaluated on objective task quality, calibration, information coverage, minority-insight retention, and robustness.

14. Current limitations

The following limitations are material and should be disclosed in any public or technical summary.

Limitation	Consequence
Multiple dialects and runtimes	A program valid in one path may not execute in another
Partially hard-coded `.hrm` behavior	Rule bodies can appear more general than the current runner actually supports
Ambiguous symbol semantics	`psi`, `phi`, and related terms vary across theoretical and numerical contexts
Coherence metric is narrow	`1 / (1 + std(phi))` measures one kind of dispersion, not truth or intelligence
Specialized simulations are not unified	Tau, quantum-inspired, lattice, cluster, and LLM results are not automatically comparable
Limited default telemetry	The basic CLI demonstrates execution but does not provide a complete experimental record
Reproducibility is not always explicit	Random initialization can change results when no seed is recorded
EEG training documentation is incomplete	The historical claim should be accompanied by data provenance and protocol details

Scientific validation is separate from software execution	Working code does not establish neuroscience, consciousness, safety, or intelligence claims
Repository maturity issues	Tests, dependencies, generated artifacts, and older compatibility paths require consolidation

These limitations do not make the project meaningless. They define the work required to turn a broad research repository into a stable language and experimental platform.

15. A concise, verbatim-ready description

> **HARMONIA DSL is an experimental symbolic and numerical language ecosystem for describing adaptive systems through coupled states such as phase or awareness, structural or ethical orientation, uncertainty, energy, memory, and coherence. Its `.hrm` files act as harmonic scores that declare systems, dimensions, laws, and constraints. The repository contains several runtimes rather than one finalized compiler, including a symbolic Φπε interpreter, a nine-layer integration stack, a declarative swarm runner, and specialized harmonic, Tau, quantum-inspired, lattice, cluster, neural, and LLM-related simulations.The harmonic swarm is a population of lightweight mathematical agents that evolve through coupled numerical dynamics. The baseline engine measures collective coherence from the dispersion of an agent state and can expose that measurement to ANIMUS, a policy layer that regulates scheduling or concurrency. The swarm is therefore best understood as a testbed for collective dynamics and adaptive governance, not automatically as a collection of conscious or language-model agents. Its value must be demonstrated through reproducible data and comparison with suitable controls.The project also includes an EEG-oriented experimental track. According to the project author, EEG data was used to train or condition the swarm, and the repository contains utilities for EEG analysis, conversion to HARMONIA representations, neural bridging, and EEG-driven swarm simulation. This establishes an EEG-informed development history, while stronger claims about neural decoding, clinical meaning, or model performance require separate documentation of the data, mapping, training procedure, and evaluation.**

16. One-sentence description

> **HARMONIA DSL is a research language and simulation ecosystem for expressing and testing how symbolic rules, coupled agent dynamics, coherence measurements,**

> **memory, and regulatory constraints interact in adaptive individual and collective systems.**

17. Evidence classifications used in this report

Classification	Meaning
Implemented	Directly represented in executable source code
Declaratively represented	Written in `.hrm` or configuration syntax, but not necessarily compiled generically
Documented intent	Described in specifications, READMEs, or theory documents
Author-provided history	Reported by the project author and preserved with attribution
Interpretation	A cautious explanation of possible use, not a verified result

References

[1] HARMONIA-DSL README
[2] HARMONIA DSL Language Reference
[3] HRM Language Specification
[4] HARMONIA .hrm Runner
[5] Pure Harmonic Swarm Engine
[6] ANIMUS Cluster Simulation
[7] Coherence-Gated Flow Simulation
[8] Symbolic Phi-Pi-E Interpreter
[9] Cluster Rescue Harmonic Score
[10] Tau Crystal Processing Layer
[11] Quantum-Inspired Swarm Engine
[12] Xi-Lambda-Omega Lattice
[13] EEG Analysis Utility
[14] EEG-to-HRM Conversion Utility
[15] EEG Swarm Simulation
[16] Neural Bridge

[17] Swarm Simulation Results